

03 — System Architecture and Implementation

SEION-KGR v26 MAX

Campaign ID: `gate12-closeout-2026-08-01`

Canonical commit: `6af3c35271ae2ffab41ecba2aad098d1988fdc0c`

Branch: `campaign/gate12-closeout`

Date: `2026-08-01`

Purpose: describe the actual code structure as it exists at the end of this campaign, module by module, with file paths.

Contents

1 Package layout	1
2 Data flow	1
2.1 Scorer composition	2
2.2 Path reasoner (Phase B2 update)	2
2.3 Structural kernel branch (Phase B3, new)	2
2.4 Certification pass (Phase B4, new)	2
3 Reproducibility engine (Phase B5 updates)	2
4 What is intentionally NOT in this architecture	2

1 Package layout

seion_kgr_reference_fp64.py	Fase 1: FP64/CPU/no-AMP independent reference oracle
seion_kgr_v26_train.py	thin CLI entrypoint -> seion_kgr.train.main()
seion_kgr/	
data.py	reciprocal KG loading, filters, negative sampling
scorers.py	DistMult / ComplEx / CP / Tucker base experts
reasoner.py	budgeted-BFS query-conditioned path reasoner
selector.py	[NEW] learned sparse path selector (Phase B2)
kernels.py	batched CP ternary law, Stiefel projector, seionic scorer
structural_kernel.py	[NEW] E8 / matched-control residual branch (Phase B3)
certification.py	[NEW] state->score->ranking certification chain (Phase B4)
projection.py	projector audit, closure-leakage measurement
rank_controller.py	6 adaptive rank-allocation policies
geometry.py	associator / Filippov-identity curriculum (weight-gated)
metaencoder.py	relation-graph co-occurrence encoder
losses.py	1-vs-all / BCE + N3 regularizer
evaluate.py	blocked filtered evaluator (reciprocal head/tail trick)
reproducibility.py	manifests, checkpoints, [NEW] config/exec identity (B5)
model.py	SeionKGRv26: wires branches behind a near-zero-init router
train.py	CLI, training loop, [NEW] out-dir guard (B5)
tests/kgr/	130 tests (66 canonical + 64 added this campaign)
.github/workflows/kgr-ci.yml	[NEW] CI (Phase B1)

2 Data flow

A training query (h, r, t) is always scored as $(h, r, ?)$ *tail*-ranking; the backward/head direction is trained and evaluated as $(t, r^{-1}, ?)$ *tail*-ranking with gold h — there is no separately implemented head-scoring code path anywhere in the system (contract §II.3.2). This is the single design decision that eliminates an entire class of duplicated-logic bugs the legacy `seion_train_v25.py` had.

2.1 Scorer composition

$$s(h, r, t) = s_{\text{base}}(h, r, t) + \gamma_r s_{\text{path}}(h, r, t) + \eta_r s_{\text{seion}}(h, r, t) + s_{\text{kernel}}(h, r, t),$$

with $\gamma_r = \sigma(\text{gamma_raw}_r)$, $\eta_r = \sigma(\text{eta_raw}_r)$ both initialized to $\sigma(-4) \approx 0.018$, and s_{kernel} 's gate internal to `StructuralKernelResidual` (same near-zero-init pattern). The model always starts as the base expert alone; every experimental branch must earn its weight.

2.2 Path reasoner (Phase B2 update)

Budgeted BFS frontier expansion, `num_layers` hops, queried-edge (and its reciprocal) excluded *before* any selector sees the candidate list — not after, so leakage prevention cannot be bypassed by selector mode. Four modes: `full_neighborhood`, `budgeted_bfs` (default, unchanged from the canonical commit), `learned_topk` (new this campaign — `LearnedPathSelector` scores each candidate edge from source state, edge relation, query relation, destination base embedding, depth, and accumulated path priority; kept edges are weighted by a differentiable $\sigma(\text{score})$ gate so gradients reach the selector), `oracle_or_gold_path_debug_mode` (synthetic-fixture-only, not CLI-exposed).

2.3 Structural kernel branch (Phase B3, new)

$$\mu_{\text{total}} = \mu_{\text{learned}} + \sigma(\epsilon_r) W(\mu_{\text{kernel}}(U_x x, U_a a, U_q q)),$$

where μ_{kernel} reuses the double-contraction construction already validated in `seion_train_v25.py`'s `fixed_predict`, against one of five frozen 248^3 tensors: `E8_exact` (real `E8_Exact_v18_2/f_E8.npy`, local-only, not committed), `random_scale_matched`, `permuted_indices`, `sign_shuffled`, `zero_kernel`. All five share an identical architecture and trainable-parameter count outside the frozen buffer (tested).

2.4 Certification pass (Phase B4, new)

A structurally separate pass from `evaluate.evaluate()` — coverage is never blended with MRR/Hits@K in the same call. `certify_evaluation_split` runs `certify_query` per test triple, reporting `certified_rank_stable_coverage` etc. separately per split.

3 Reproducibility engine (Phase B5 updates)

Every run directory gets: `resolved_config.json`, `command.txt`, `environment.json`, `hardware.json`, `git_manifest.json`, `dataset_manifest.json`, `run_manifest.json` (now carrying `configuration_id/execution_id/parent_execution_id/resume_from`), `metrics.jsonl`, `rank_history.jsonl`, `error_attribution.jsonl`, `projection_audit.json` (when the path branch is enabled), `kernel_manifest.json` (when the structural-kernel branch is enabled), `best.pt/last.pt` (each carrying the same identity triple), `final_metrics.json`. An out-dir reuse guard rejects a non-empty `-out_dir` unless `-resume` is set; a resume whose `configuration_id` does not match the checkpoint's is rejected before `load_state_dict`, not after.

4 What is intentionally NOT in this architecture

No global attention mechanism; no learned graph-level search beyond the per-query budgeted selector; no multi-dataset joint pretraining loop (the relation metaencoder exists as a library module, unit-tested, not wired into `train.py`'s CLI); no mixed-precision training loop in `train.py` itself

(BF16/FP32 parity is verified at the kernel level in `tests/kgr/test_perf_and_parity.py` on the real GPU, but the trainer does not yet expose an `-amp` flag — an honest, explicit gap, not a silent one).